PROJECT GUIDE

OptiChain

Demand forecasting and stock planning for a real retail chain, explained from A to Z: what was done, what every number means, and why each choice was made.

Saved vs the official method	Over a year (estimate)	Stock on shelves	Store-products planned
€708	**≈ €6,100**	**−45%**	**599**
in 6 weeks · 16% cheaper	if the test weeks are typical	1,487 vs 2,713 units a week	67 stores · 297 products

Contents

A · The problem

B · The data

C · The pipeline

D · The 20 steps

E · The dashboard

F · The code

G · Results

H · Limitations

I · Questions and answers

J · How to run

Data: Vandeput, N. (2025). *VN2 Inventory Planning Challenge*. Organised by Nicolas Vandeput (SupChains); sponsored by Flieber, Daybreak and SupChains. https://supchains.com/vn2-inventory-competition/ · Licence CC BY 4.0.

A

The problem we are solving

A shop chain must decide **every Monday how much stock to order** for each product in each shop. The order takes **two weeks** to arrive. Two mistakes cost money:

Mistake	What happens	Cost (official rule)
Order too little	The shelf goes empty and the customer leaves	**€1 per missed sale**
Order too much	Boxes sit unsold on the shelf	**€0.20 per unit per week**

Goal: predict demand, then order the amount that makes the total cost as small as possible.

> **Why this matters.** Retailers lose money both ways: empty shelves lose customers, extra stock ties up money and space. Balancing the two is the core problem of supply-chain inventory planning.

B

The data: the VN2 Inventory Planning Challenge

What it is. A real competition held in 2025 by Nicolas Vandeput (SupChains), with about 180 teams. The data is **real weekly sales** of an anonymous retail chain, published on GitHub (SupChains/VN2).

Why we chose it over Walmart M5 (the project's first version, kept in *archive/m5*):

- M5 is older (2011-2016) and has **no stock data**, so stock levels and costs had to be invented.
- VN2 is recent (2021-2024) and comes with **real stock, real costs, real rules** and an official score to compare against.
- It is free to use (CC BY 4.0) as long as the competition is cited.

Size of the data

Item	Value	Meaning
Stores	67	shops in the chain
Products	297	different items sold
Store-products	599	one product in one store; not every store sells every product. **Each is planned separately.**
Weeks	165	2021-04-12 to 2024-06-03
History	157 weeks	up to week 0 (8 April 2024), used for learning
Competition	8 weeks	weeks 1-8, used **only** for the final test

The files

File	What is inside
Sales (Week 0 ... Week 8)	units sold per store-product per week; each file adds one week
In Stock	True/False per week: was stock left at the end of the week?
Master	groupings: product group, department, store format
Initial State	the real stock on the shelf and orders on their way at the start

File	What is inside
Benchmark orders	the official simple method's six orders, used to verify our simulator

Why stores and products are numbers. The company hid its identity. The dashboard therefore gives five example products descriptive nicknames ("The Christmas Rocket") based on how they behave, while always showing the real code next to them. Nothing is invented.

Data: VN2 Inventory Planning Challenge (Vandeput, 2025), CC BY 4.0

C

The whole pipeline in one picture

Step	What happens
1	Load the data and check its quality
2	Explore it with charts and statistics
3	Find and handle hidden missing weeks
4-11	Classical time-series analysis: stationarity, seasonality, ACF/PACF, ARIMA, AIC/BIC, residual tests, GARCH
12-13	Ten forecasting methods, tested fairly on 30 past weeks
14	Measure how uncertain each forecast is
15	Simulate the shop with the official rules
16	Decide how much to order (Monte Carlo newsvendor)
17	Final test on the 8 real competition weeks
18	Recommend next week's order
19-20	Save results for the dashboard and conclude

Everything lives in **one notebook**, *notebooks/OptiChain_VN2_Pipeline.ipynb*, in 20 explained steps. The dashboard only displays its results, except the stock simulation, which it re-runs live when you change a setting.

D

The 20 steps, one by one

Step 1 · Data collection and audit

- **What:** download the files if missing, then check: no empty cells, no negative sales, only whole numbers, and all nine sales files agree with each other.
- **Why:** if the data is broken, every later result is wrong.
- **Found:** 43% of weeks have zero sales (most products sell slowly); about 1 week in 10 is flagged out of stock.

Step 2 · Exploratory data analysis (EDA)

Looking at the data before modelling. Each chart answers one question:

Chart	Question	What we found
Line graph of total sales	Is there a trend or season?	Stable level with a strong Christmas peak each year
Seasonal profile	Which weeks of the year sell more?	Weeks 48-52 sell up to 2.4× an average week
Histograms	How much does a typical product sell?	Median about 1 unit a week; very skewed
Pareto curve	Where is the volume?	A few products make most of the sales
Scatter plots	Volume vs variability; this week vs last week	Slow sellers are most erratic; this week follows last week
QQ plots	Are sales normally distributed?	No: heavy tails, especially single products
Descriptive statistics	Mean, spread, skewness, kurtosis	Strong right skew, many zeros; Jarque-Bera rejects normality
Stockout charts	Which stores run out?	From 0% to 59% of weeks, depending on the store

Why it matters: slow, zero-heavy demand needs special methods (Croston, TSB), and non-normal data needs a count distribution rather than a bell curve. The EDA decided both.

Step 3 · Missing values

The file has no empty cells, but two kinds of weeks are **hidden missing values**:

- **Before launch.** A product shows 0 before it was ever sold in that store. Those zeros are not "nobody wanted it": the product did not exist yet. Every week before the first sale is treated as missing.
- **Out-of-stock weeks. 100% of out-of-stock weeks recorded zero sales**: the shelf was empty, so nothing could sell. That zero measures supply, not demand (statisticians call it *censored demand*), so it is treated as missing.

Why not fill the gaps with zero or a guess? A zero would teach the models that demand is lower than it is, so they would under-order; a guess would look like real information. Each method handles the gap honestly:

Method	Treatment of a missing week
Averages, smoothing, Croston, TSB	skip it
ARIMA / SARIMA	kept missing: the Kalman filter skips it exactly
ADF, KPSS, STL, ACF (need an unbroken series)	straight-line interpolation between the neighbouring weeks
LightGBM	never learned as a target

Demand patterns. Each store-product is also labelled by two numbers: **ADI** (average weeks between sales) and **CV²** (how much sale sizes vary). Together they give four types: **steady, spiky, occasional, unpredictable** (Syntetos-Boylan classification).

Step 4 · Stationarity (ADF test)

- **Stationary** means the average and the spread do not drift over time. ARIMA-type models need it.
- **ADF** asks "does the series wander like a random walk?" A p-value below 0.05 means stationary. **KPSS** asks the opposite question, as a second opinion.
- **Result:** the network total is stationary by ADF; **94% of the 599 store-products** are stationary.

Step 5 · Transformations

If a series is not stationary, transform it: a **log** (calms a growing spread), a **difference** (this week minus last week, removes drift) or a **seasonal difference** (minus the same week last year). All were tested; **no differencing was needed (d = 0)**, so the original units were kept, which is also what ordering needs.

Step 6 · Seasonality (STL decomposition)

STL splits the series into **trend** (slow movement), **season** (the yearly repeat) and **remainder** (noise). Seasonal strength **0.93** on a 0-1 scale: very seasonal (Christmas). Most single products are too noisy and too short to estimate their own season, so the models take the season from the whole network and apply it to each product.

Step 7 · ACF and PACF

- **ACF**: how strongly this week relates to k weeks ago.
- **PACF**: the same, keeping only the direct link (the weeks in between removed).
- Found: strong links at lags 1-5, and around lag 52 (one year back).

Step 8 · Identify the model

What the plots show	Model
PACF cuts off, ACF fades	**AR**: depends on past values
ACF cuts off, PACF fades	**MA**: depends on past errors
Both fade	**ARMA**
Differencing was needed	**ARIMA**
Spike at the seasonal lag (52)	**SARIMA**

For the network total: an AR pattern plus a yearly spike, so SARIMA.

Step 9 · Lags

The useful lags are **1-5** (recent weeks) and **52** (the same week last year). This also chose the inputs of the machine-learning model in step 12.

Step 10 · AIC and BIC

- **AIC** = fit quality minus a penalty for complexity; **BIC** = the same with a stricter penalty. Lower is better.
- **Network total:** 27 candidates compared; the winner is **SARIMA(2, 0, 0)x(1, 0, 0)[52]** (BIC prefers SARIMA(1, 0, 0)x(1, 0, 0)[52]). This week depends on the last two weeks and on the same week last year.
- **Every store-product** gets its own ARIMA: ADF picks d, then AIC picks p and q. Result: 38% ARMA, 20% AR, 9% MA, 5% ARIMA, and 28% "mean only" (slow sellers with so little signal that a plain average wins).

Step 11 · Residual diagnostics, ARCH and GARCH

Residuals are what the model could not explain. A good model leaves only random noise (**white noise**).

Test	Question	Rule
Ljung-Box	Is any pattern left?	p above 0.05: white noise
Jarque-Bera	Are the residuals normal?	p above 0.05: normal
ARCH-LM	Do calm and turbulent periods alternate?	p below 0.05: yes, GARCH is needed

Network total	SARIMA residuals	After GARCH(1,1)
Ljung-Box p (lag 8)	0.012	0.051
Ljung-Box p (lag 13)	0.116	0.251
ARCH-LM p	0.027	0.955
White noise?	no	yes
ARCH effect?	yes	no

- The SARIMA residuals showed an **ARCH effect**: the noise is bigger in some periods (around Christmas).
- **GARCH(1,1)** models that changing noise level. After it, Ljung-Box passes and the ARCH effect is gone.
- Normality is still rejected because of a few extreme weeks. That does not bias the forecast; it only affects how wide the intervals should be.
- **Test on unseen weeks:** the SARIMA forecast the 8 competition weeks with **14% error**, against 20% for "same week last year", and every real week fell inside its 80% range.
- **All 599 store-products:** 91% have white-noise residuals; 10% showed an ARCH effect, and GARCH removed it in 85% of those.

Step 12 · Ten forecasting methods

Ten methods are compared so that the choice is proven, not assumed.

Method	Idea	Why include it
Last week	Next week = last week	Simplest baseline

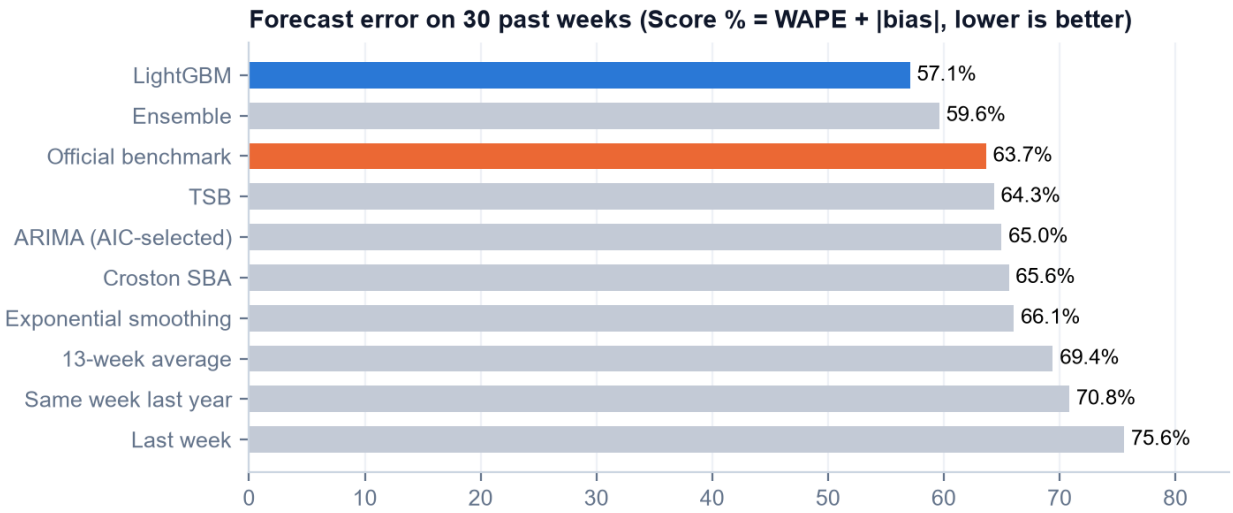
Method	Idea	Why include it
13-week average	Average of the last three months	Simple and stable
Same week last year	Copy last year	Captures the season simply
Official benchmark	13-week average × seasonal factor	The competition's own method: the target to beat
Exponential smoothing	Weighted average that trusts recent weeks more	Classic statistical method
ARIMA	The Box-Jenkins model of steps 4-11, one per store-product	The classical time-series approach
Croston SBA	Predicts sale size and gap between sales separately	Built for slow sellers
TSB	Like Croston, but notices products that stop selling	Built for slow sellers
LightGBM	One machine-learning model trained on all 599 at once	Learns shared patterns; the VN2 winners used this type
Ensemble	Average of LightGBM, smoothing and the benchmark	Combining usually reduces error

LightGBM inputs: the last 8 weeks of demand, rolling averages, the same week last year, the season factor, in-stock flags, weeks since launch, and the store and product groups. Its loss function is **Tweedie**, designed for data with many zeros; it beat the alternatives in a test on past weeks.

Step 13 · Backtest: testing on the past, fairly

- At **30 past weeks**, each method sees **only the data before that week** and predicts the next three weeks; the prediction is then compared with what really sold.
- **Why three weeks?** An order placed now arrives in two weeks, so the planner needs this week, next week and the arrival week.
- **Why only past data?** Using future data would be cheating (*leakage*) and make results look better than reality.

Metric	Meaning
MAE	average miss, in units
WAPE	total error ÷ total sales
Bias	do we guess too high (+) or too low (−) overall?
Score = WAPE + |Bias|	the official metric of the VN1 forecasting competition; lower is better



Why do errors look large (about 60%)? Most products sell 1-2 units a week: guessing 2 when 1 sells is already "100% wrong". Errors for slow sellers always look big in percentage terms.

Ljung-Box on the real forecast errors: for most methods, 88%-93% of store-products leave no pattern in their mistakes; "Last week" only 52%. A pass shows that no pattern is left; it does not by itself prove accuracy, which the leaderboard measures.

Step 14 · Uncertainty: how wrong can a forecast be?

- **Why:** ordering exactly the average runs out about half the time. The spread matters.
- **What:** demand is modelled as a **negative binomial distribution**, which suits counts (whole numbers, many zeros, occasional spikes). Its mean is the forecast; its spread **k** is learned per product from past errors.
- **Calibration check:** when the model says "80% chance demand stays below X", it does about 85% of the time: slightly cautious, which step 16 corrects.

Step 15 · The inventory simulator

Our code plays the exact official rules every week:

```
stock at start = last week's leftover + the arriving order
sales = smaller of (stock, demand)
missed sales = demand − sales (lost, no back-orders)
cost = €0.20 × leftover + €1 × missed sales
```

> **Verified.** With the organiser's own benchmark orders, our simulator gives exactly **€4,334.0**, the official number, and our rebuild of the benchmark places the same orders. Every score in this project is therefore directly comparable with the real leaderboard.

Step 16 · The ordering policy: how much to order

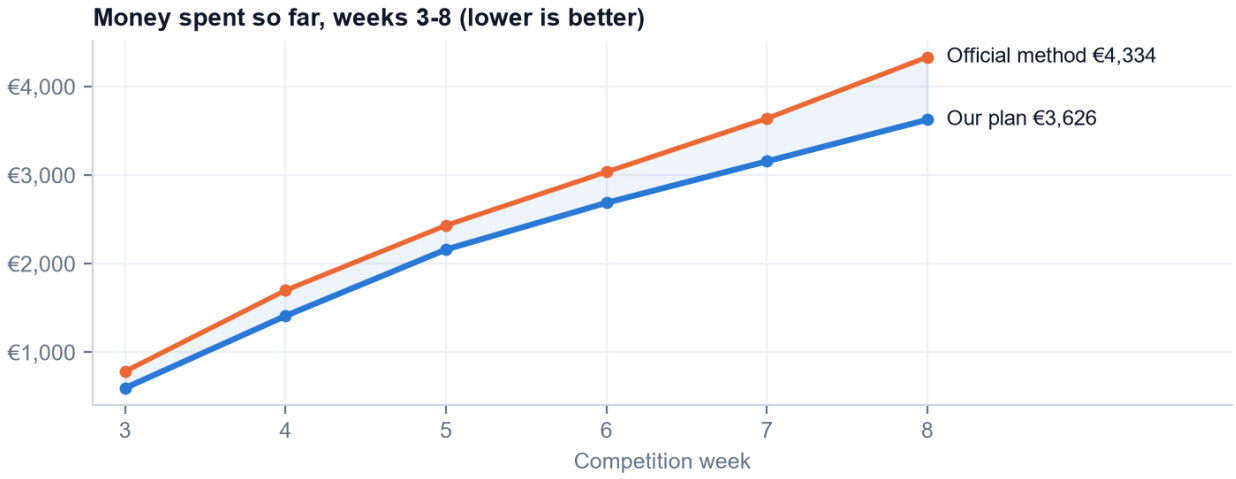
Monte Carlo newsvendor, for each store-product, every week:

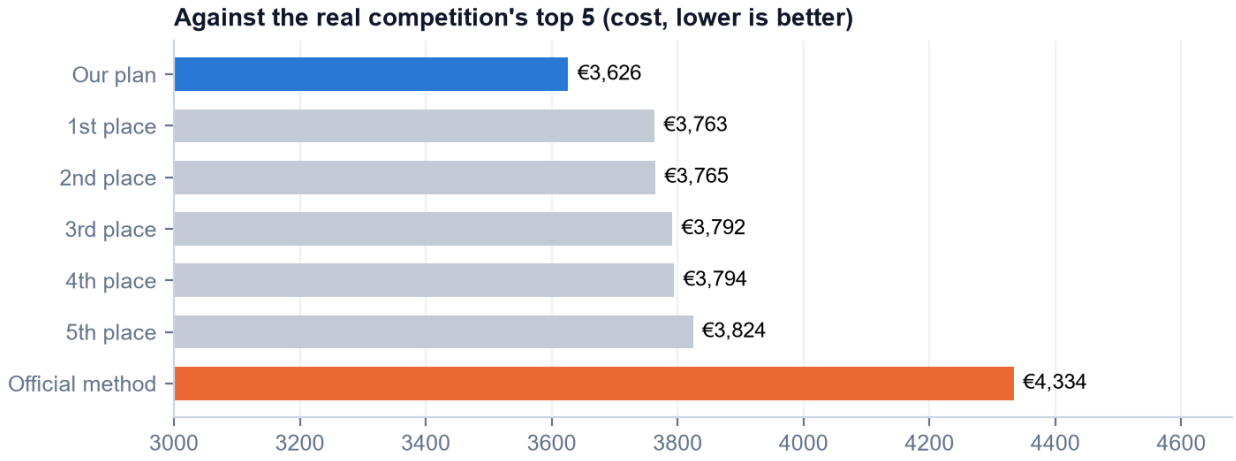
- Simulate **1,000 possible futures** of demand for the next three weeks.
- In each future, play the two weeks before the order arrives, to see how much stock will be left.
- Need = demand in the arrival week − stock left.
- Order enough to cover that need in a chosen share of futures: the **service level**.

The textbook service level is 1 ÷ (1 + 0.2) = **0.833**. Instead, the level was **tuned by replaying 18 past weeks**: **0.75** gave the lowest cost. It sits below the textbook value because the spread estimate is slightly cautious and leftover stock is not wasted (it sells next week). Simulation captures both, which is what the competition's top teams did.

Step 17 · Final test: the eight real competition weeks

Each week the models were retrained on the data known up to that week; the service level was tuned on the past, **never on these weeks**.

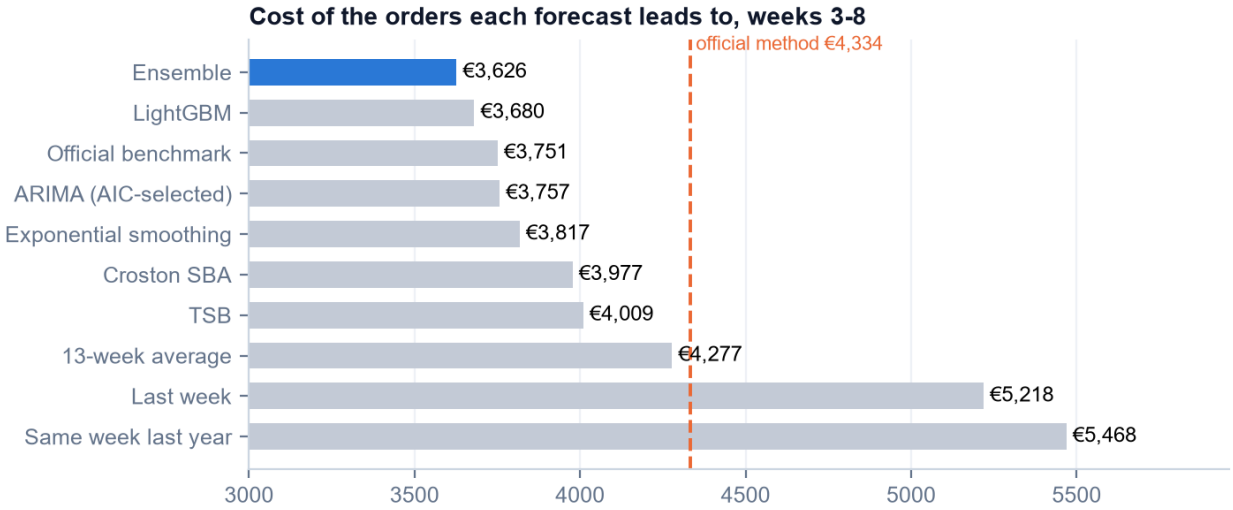




Plan	Cost, weeks 3-8
Our plan (Ensemble @ 0.75)	**€3,626**
LightGBM plan	€3,680
ARIMA plan	€3,757
Real 1st place	€3,763
Official benchmark	**€4,334**

> **Two honest points.** (1) Hindsight: this was built after the competition, knowing which methods worked, so "we beat the official method by 16%" is fair, "we beat 1st place" is not fully fair. (2) On these 8 weeks the simple benchmark *forecast* was slightly more accurate, yet our *orders* still cost less: short test windows shuffle rankings.

The trade-off: the plan keeps **45% less stock** on the shelves (1,487 vs 2,713 units a week) and accepts more missed sales (1,842 vs 1,078 units). Under the €1 vs €0.20 rules, that is the cheaper deal overall.



Step 18 · Stock outlook: the next order

The data ends on 3 June 2024, so the plan treats **10 June 2024 as "today"**: it orders **1,162 units for 271 store-products**, arriving on 24 June (two weeks later), and flags which products might run out. Today's real date cannot be used: there is no data after June 2024, so a plan for today would be invented.

Steps 19-20 · Save and conclude

All results are saved to *artifacts/*, which the dashboard reads; the notebook ends with a summary and its limitations.

E

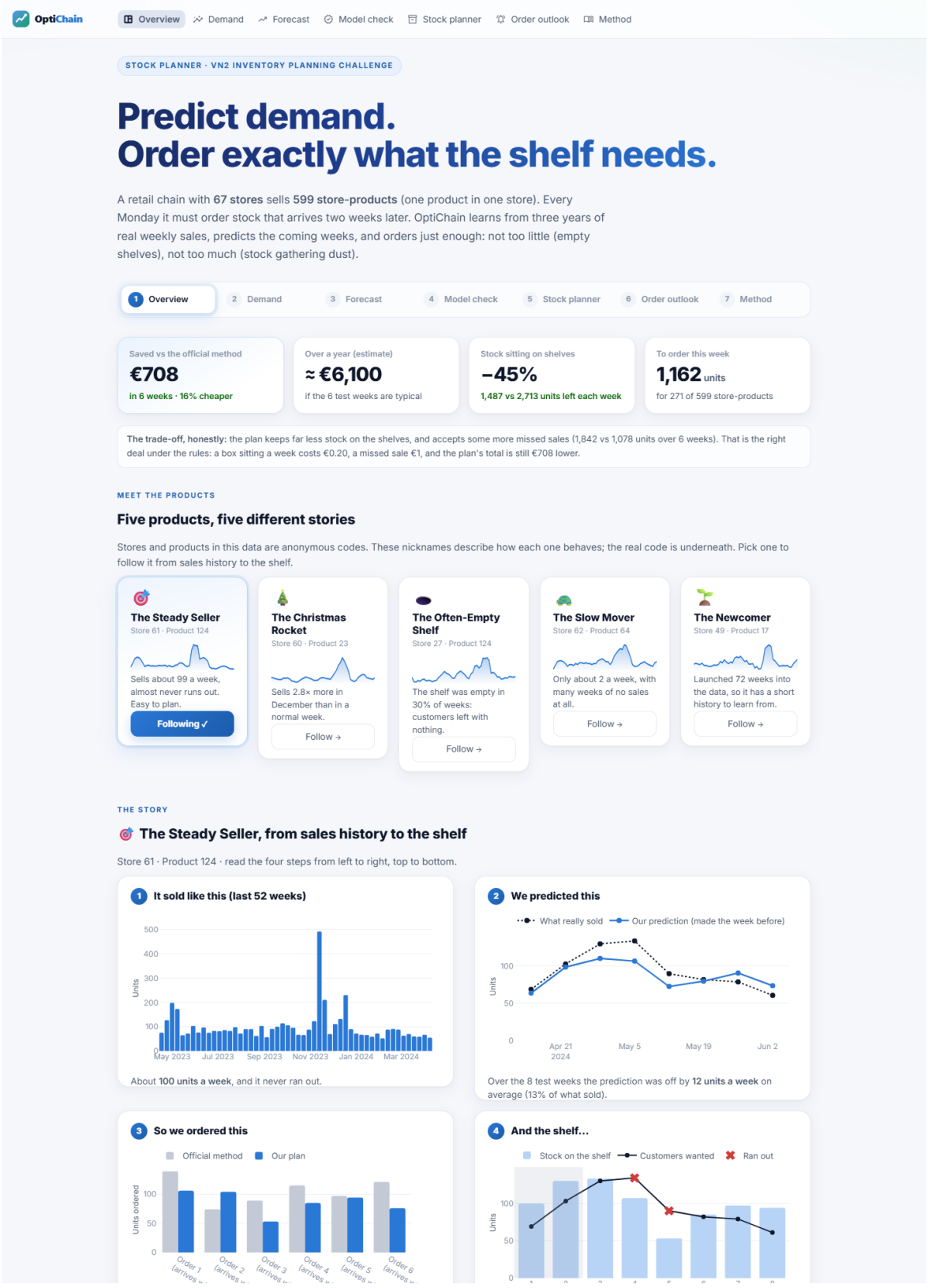
The dashboard: seven pages

Page	Question it answers	What is on it
Overview	Did it work?	Money headlines (saved, per year, stock on shelves, units to order); five hero products; a four-step story per product (sold, predicted, ordered, shelf result); the result against the competition.
Demand	How does a product sell?	Sales history (orange bands = empty shelf), yearly seasonality, a map of sales patterns, stockouts by store and month.
Forecast	Which method predicts best?	Ranking of the 10 methods, a "try it" forecast for any product and date with an 80% range, error by pattern, LightGBM's most important inputs.
Model check	Are the statistics sound?	Tabs for ADF stationarity, STL season and trend, ACF/PACF, AIC/BIC model choice, residuals and GARCH, forecast check; for the whole network or live for any product.
Stock planner	What if I change the settings?	Choose a method and a service level; the eight weeks replay live; a cost curve shows the sweet spot; a week-by-week drill-down; download the orders.
Order outlook	What do I order this week?	The order total, a risk chart, every store-product with the riskiest first, CSV download.
Method	How was it built?	The rules, a glossary, calibration, limitations and the data citation.

Design choices: blue is always our plan, orange the official method, grey is context; every chart has a plain-language note; the palette is colour-blind safe.

Dashboard · Overview

Did it work?



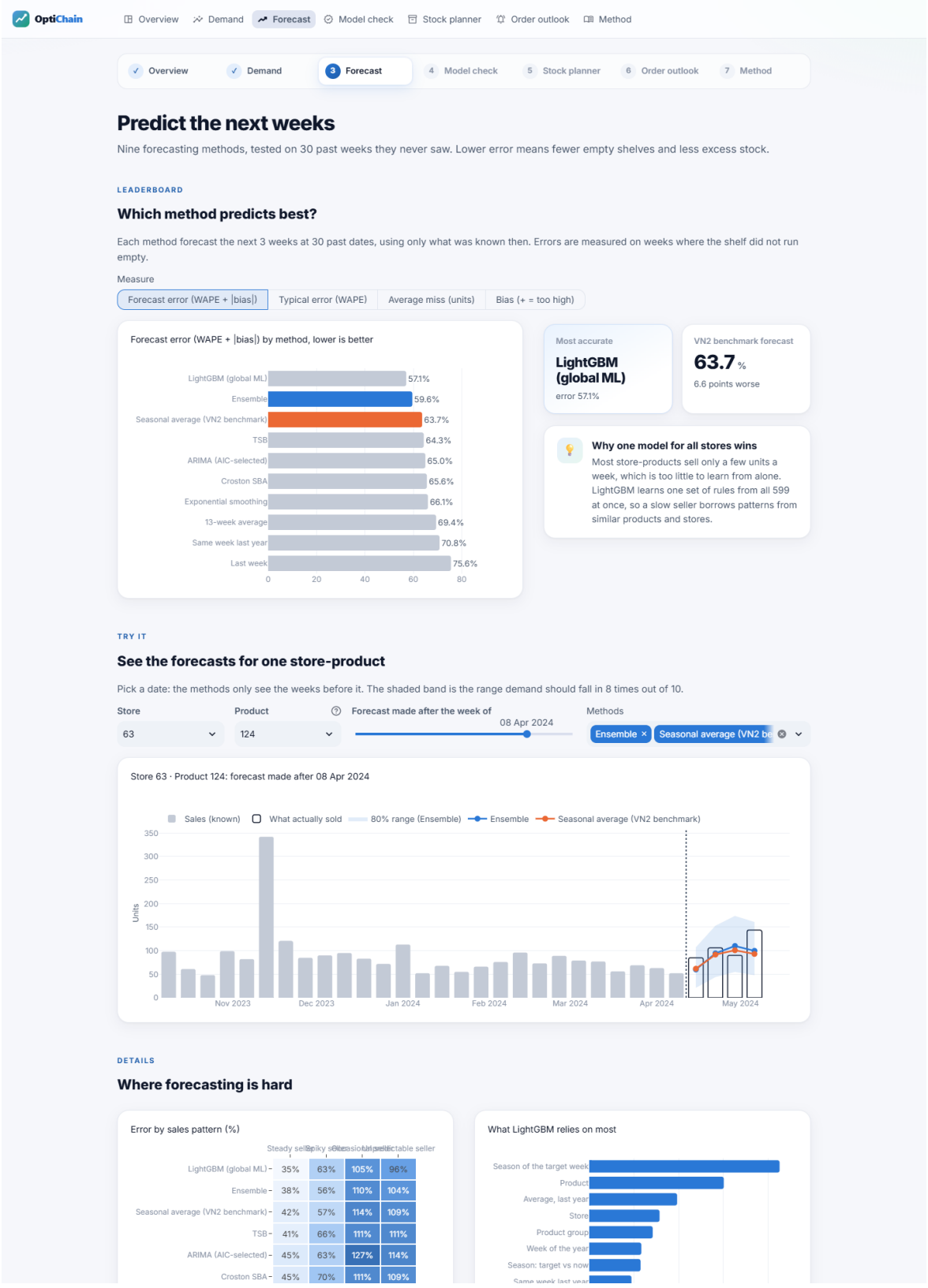
Dashboard · Demand

How does a product sell?



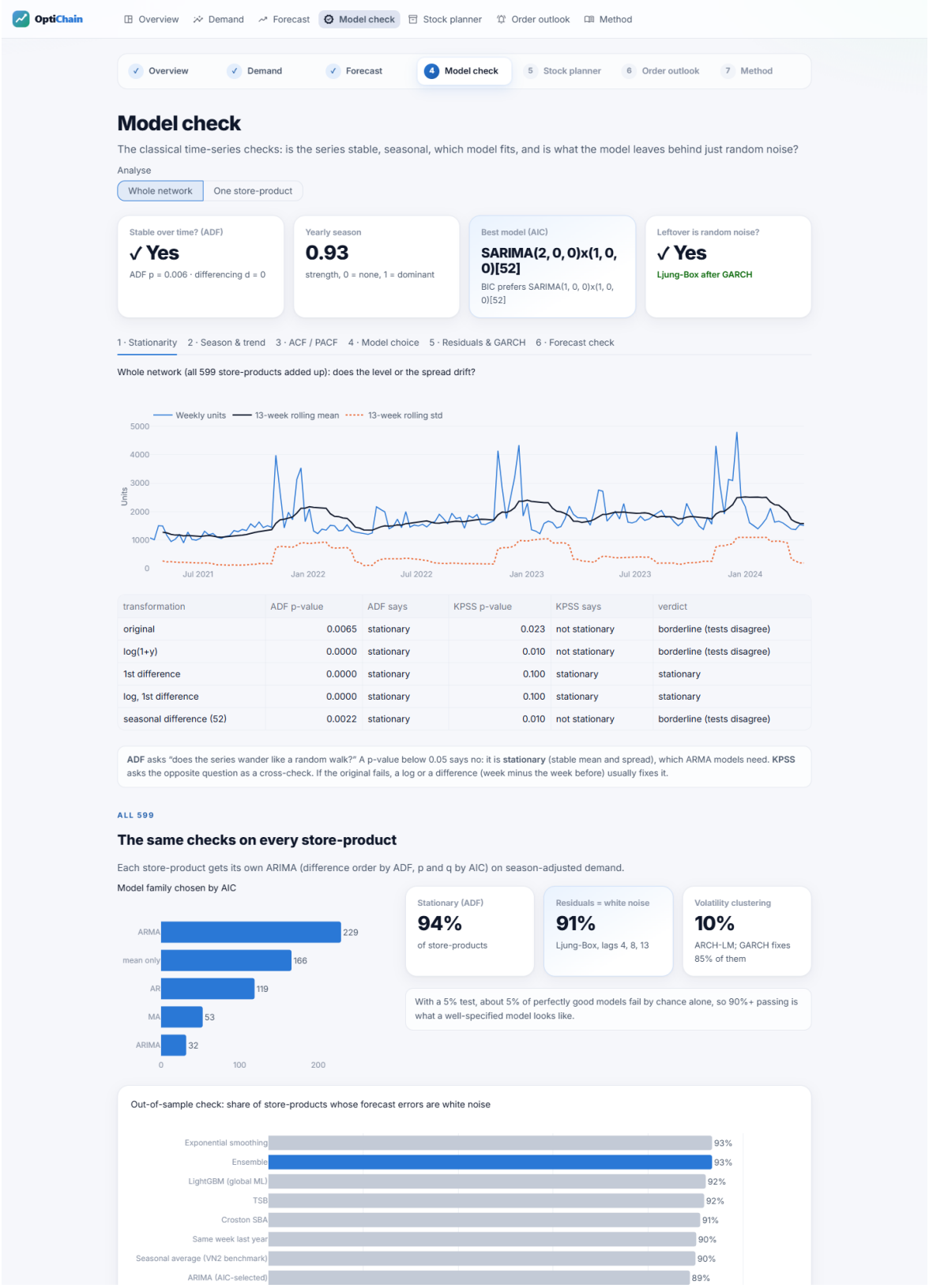
Dashboard · Forecast

Which method predicts best?



Dashboard · Model check

Are the statistics sound?



Dashboard · Stock planner

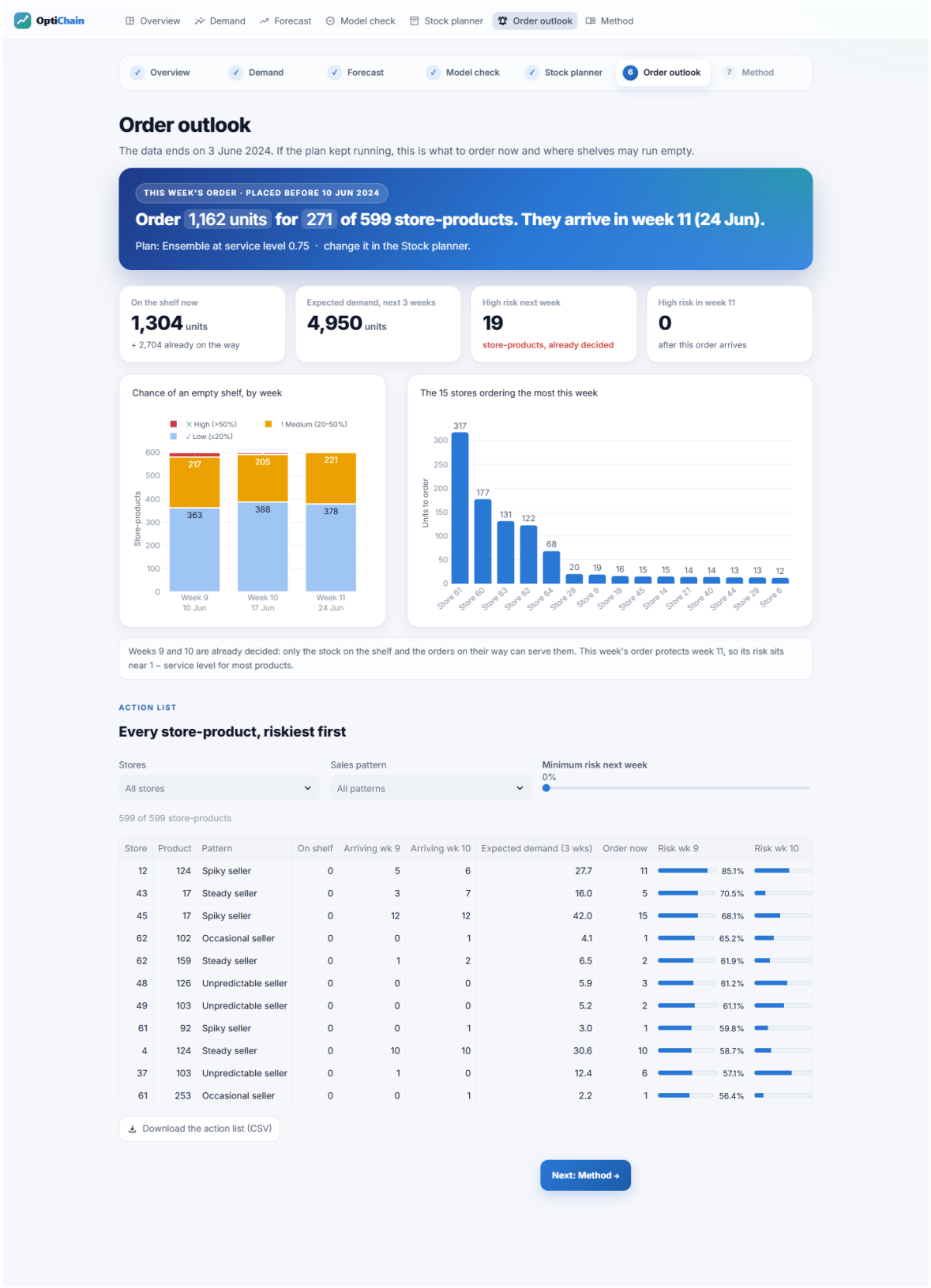
What if I change the settings?



week	stock at start	demand	sold	missed	left at end	order placed	arrives
1	83	85	83	2	0	115	week 3
2	83	106	83	23	0	115	week 4
3	115	90	90	0	25	67	week 5
4	140	144	140	4	0	60	week 6
5	67	101	67	34	0	111	week 7
6	60	82	60	22	0	80	week 8
7	111	70	70	0	41	0	
8	121	65	65	0	56	0	

Dashboard · Order outlook

What do I order this week?



Dashboard · Method

How was it built?



F

The code: where everything lives

File	Job
config.yaml	all settings in one place: costs, models, test windows
src/optichain/data.py	load files, find launches and stockouts, classify demand patterns
src/optichain/timeseries.py	ADF, KPSS, STL, ACF/PACF, AIC/BIC, Ljung-Box, ARCH, GARCH, per-product ARIMA
src/optichain/models.py	the forecasting methods, including LightGBM
src/optichain/evaluation.py	backtest and accuracy metrics
src/optichain/inventory.py	official simulator, demand sampling, newsvendor orders
src/optichain/planning.py	weekly rounds and the live plan
scripts/build_notebook.py	writes the notebook (edit this, not the .ipynb)
scripts/build_guide.py	writes this PDF
app/	the dashboard: Home.py, one file per page, ui.py design system, data.py loaders
tests/	automatic tests, including "the simulator gives exactly €4,334"
archive/m5/	the earlier Walmart version, kept, not deleted

G

Results at a glance

Question	Answer
Best forecaster (30 past weeks)	LightGBM, error 57.1% (official benchmark 63.7%)
Best plan (8 real weeks)	Ensemble at service level 0.75: €3,626
Saving vs the official method	€708 (16%) in 6 weeks; ≈ €6,100 a year
Network time-series model	SARIMA(2, 0, 0)x(1, 0, 0)[52]; residuals white noise after GARCH(1,1)
Per-product ARIMA residuals	91% white noise (Ljung-Box)
Simulator check	reproduces the official €4,334.0 exactly

H

Limitations

- **Hindsight:** the design was informed by the published competition results.
- **Censored demand:** the simulation uses recorded sales as demand, so weeks that ran out in reality understate true demand (the competition used the same convention).
- **Short test:** eight weeks is short; a few big products can move the score.
- **Anonymous data:** what the products really are is unknown.
- **Simple uncertainty model:** quantile models could calibrate the risk ranges better.

I

Questions you might be asked

Question	Short answer
Why VN2 and not Walmart?	It is recent, real, and has real stock levels, real costs and an official score to compare against.
Why remove out-of-stock weeks?	All of them show zero sales: those zeros measure an empty shelf, not customer demand.
What does "store-product" mean?	One product in one store. 67 stores × 297 products, but only 599 pairs exist; each is planned separately.
Why is ARIMA not the final model?	It was tested fairly: LightGBM and the ensemble were more accurate and cheaper. ARIMA still beat the official method.
What does Ljung-Box prove?	That no pattern is left in the errors. Accuracy is proven separately, on unseen weeks.
Why GARCH?	The noise was bigger in some periods (Christmas). GARCH models that, and the tests then pass.
Why service level 0.75 and not 0.833?	It was tuned by simulating real costs on past weeks, which beats trusting the formula.
Why does the plan miss more sales than the official method?	Because a missed sale costs €1 but a leftover costs €0.20 a week; keeping 45% less stock is worth some extra misses, and the total is lower.
Why does the dashboard say 24 June 2024?	That is the first planning week after the data ends; an order placed on 10 June arrives two weeks later.
Is the comparison with 1st place fair?	Not fully: the project was built after the competition. The fair claim is the 16% saving over the official method.

J

How to run it

```
python -m venv .venv
.venv/Scripts/python -m pip install -r requirements.txt
.venv/Scripts/python -m pip install -e .

python scripts/build_notebook.py --run   # the whole pipeline, about 20 minutes
streamlit run app/Home.py              # the dashboard, http://localhost:8501
python -m pytest                       # the tests
python scripts/build_guide.py          # this PDF
```